People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research



National Higher School of Computer Science

Sidi Bel Abbès

Low-Cost HPC Cluster
using MPI for
Real-Time Violence Detection

Project Members

- BELAID El Baraa
- BOUCHEMELLA Mohamed
- TOUATI Tliba Mohamed Sghir
- BLIZAK Haithem
- BENGHERABI Mohamed Amine

Supervisors

- Dr Abdelkader AMRANE
- Dr Abdelatif RAHMOUN

Option

ISI

Academic Year 2025–2026

Contents

1 Cluster Connectivity **4**
1.1 Overview . . . 4
1.2 Network Architecture . . . 4
1.2.1 Subnet Layout . . . 4
1.2.2 Internet Sharing via NAT . . . 4
1.3 SSH — Passwordless Authentication . . . 5
1.3.1 Automated Key Generation . . . 5
1.3.2 Fixing SSH MITM Errors . . . 5
1.4 MPI Connectivity . . . 6
1.4.1 MPI Run Commands . . . 6
1.4.2 Automated MPI Installation on All Nodes . . . 6
1.4.3 Distributing Source Code to All Nodes . . . 6
1.5 Dynamic MPI Receive Pattern . . . 7
1.6 Protocol Summary — Connectivity Layer . . . 7

2 Network Booting (PXE / NFS) **8**
2.1 Overview . . . 8
2.2 PXE Boot Process . . . 8
2.2.1 Boot Sequence Step by Step . . . 8
2.2.2 Verifying Bootloader Configuration . . . 8
2.3 Host Setup — Dependencies . . . 9
2.4 NFS — Root Filesystem Export . . . 9
2.5 DHCP + TFTP — dnsmasq Configuration . . . 9
2.6 Golden Image & Auto-Provisioning . . . 10
2.6.1 Auto-Provision Script . . . 10
2.6.2 Enable SSH on First Boot . . . 11
2.7 Protocol Summary — Network Boot Layer . . . 11

3 Cluster Topology **11**

4 Raw Benchmark Results **12**

5 Per-Worker Performance (4 Pis) **12**

6 Communication Analysis **12**

7 Parallel Performance Metrics **12**
7.1 Speedup . . . 12
7.2 Efficiency . . . 13

8 Amdahl's Law Analysis **13**
8.1 Concept . . . 13
8.2 Amdahl's Speedup Formula . . . 13
8.3 Application to This Cluster ($p = 4$) . . . 14
8.4 Infinite Processor Limit . . . 14

8.5 Comparison with Measured Results . . . 14
8.6 Sources of Overhead . . . 14
8.7 Key Engineering Insight . . . 14
8.8 Intuitive Interpretation . . . 15

9 Gustafson's Law (Scaled Speedup Model) 15
9.1 Motivation and Concept . . . 15
9.2 Amdahl vs. Gustafson Paradigms . . . 15
9.3 Mathematical Model . . . 15
9.4 Application to This Cluster . . . 16
9.5 Final Interpretation . . . 16

10 Karp–Flatt Metric (Experimental Serial Fraction) 16
10.1 Concept . . . 16
10.2 Formula . . . 16
10.3 Application to This Cluster . . . 16
10.4 Interpretation . . . 17

11 Ideal Case and What-if Scenarios 17

12 Speedup vs. Theoretical Models 17
12.1 What the Plot Represents . . . 17
12.2 Key Insight . . . 18
12.3 Speedup Plot . . . 18

13 Comparative Performance Analysis 19

14 Proposed Network Upgrade and Recomputed Performance 19
14.1 Current Bottleneck . . . 19
14.2 Proposed Solution . . . 20
14.3 Recomputed Communication Time . . . 20
14.4 Recomputed Total Runtime and Speedup . . . 20
14.5 Recomputed Amdahl Analysis . . . 20
14.6 Comparison Table . . . 20

15 Key Conclusions 21

16 Violence Detection — Introduction & Motivation 21
16.1 Introduction . . . 21
16.2 Motivation . . . 21

17 Dataset 22
17.1 Description . . . 22
17.2 Characteristics . . . 22

18 Data Preprocessing Pipeline 22
18.1 Configuration and Data Split . . . 22
18.2 Frame Extraction . . . 23
18.3 Caching for Performance . . . 23

19 Deep Learning Models **23**
19.1 Baseline: ResNet-Based Model 23
19.2 Hybrid Model: R3D-18 + Transformer (Final Model) 23

20 Evaluation and Results **24**

21 MPI Integration on the Raspberry Pi Cluster **24**
21.1 Architecture 24
21.2 Key Constants 25
21.3 Frame Sampling 25
21.4 MPI Tensor Transfer 25
21.5 Master Workflow 26
21.6 Worker Workflow 26

22 Streamlit Application **27**

23 Conclusion **27**

1 Cluster Connectivity

1.1 Overview

The connectivity layer ties together all Raspberry Pi nodes into a functional cluster, allowing them to **communicate with each other**, share workloads via MPI, and reach the internet through the host machine acting as a gateway.

Key Goals

- Establish a private subnet between the host and all Pi nodes.
- Enable internet access on Pi nodes via NAT masquerading.
- Configure SSH for both interactive access and MPI job launching.
- Automate repetitive network configuration tasks across all nodes.

1.2 Network Architecture

Protocols & Technologies Used

- **NAT (Network Address Translation)** — host shares its uplink to Pi nodes.
- **IP Forwarding** (`net.ipv4.ip_forward`) — kernel-level packet routing.
- **iptables** — firewall rules for masquerading and forwarding.
- **Static IP addressing** — each node has a fixed IP in `192.168.50.0/24`.
- **dnsmasq** — DHCP and DNS resolution for the cluster subnet.

1.2.1 Subnet Layout

Node	IP Address	Role
Host machine	`192.168.50.1`	Gateway / NFS server / MPI master
Pi node 0	`192.168.50.100`	Compute node
Pi node 1	`192.168.50.101`	Compute node
Pi node 2	`192.168.50.102`	Compute node
Pi node 3	`192.168.50.103`	Compute node
Pi node 4	`192.168.50.104`	Compute node

1.2.2 Internet Sharing via NAT

The host machine acts as an internet gateway. IP forwarding is first enabled at the kernel level, then `iptables` rules masquerade outgoing traffic and allow forwarding between the cluster interface and the uplink:

Listing 1: Enable IP forwarding and configure NAT

```
# Enable kernel IP forwarding (persists until reboot)
sudo sysctl -w net.ipv4.ip_forward=1

# Masquerade outgoing traffic on the uplink interface
sudo iptables -t nat -A POSTROUTING -o enx022a010e1b03 -j
   MASQUERADE

# Allow forwarding from cluster NIC to uplink NIC
sudo iptables -A FORWARD -i enx5c53102a71f8 -o enx022a010e1b03
   -j ACCEPT
```

1.3 SSH — Passwordless Authentication

SSH is the backbone of cluster management. Every management task (file copy, remote command, MPI job launch) runs over SSH, so password prompts must be completely eliminated via key-based authentication.

Key Points

- Each Pi node generates its own **Ed25519 key pair** (faster and more secure than RSA-2048).
- Public keys are distributed to all other nodes via `authorized_keys`.
- OpenMPI uses SSH internally to spawn remote `orted` daemons — passwordless access is **mandatory** for MPI to work.
- Known-hosts conflicts (MITM warnings after re-imaging) are resolved by removing stale fingerprints.

Protocol Used

SSH (Secure Shell) — encrypted remote shell protocol.
Authentication mode: **public-key (Ed25519)**, no password prompt.

1.3.1 Automated Key Generation

The loop below generates a fresh Ed25519 key pair on every node in a single command issued from the host:

Listing 2: Generate SSH keys on all Pi nodes automatically

```
for ip in 100 101 102 103 104; do
    ssh pi@192.168.50.$ip \
        "ssh-keygen -t ed25519 -N '' -f ~/.ssh/id_ed25519"
done
```

1.3.2 Fixing SSH MITM Errors

When a node is re-imaged its host key changes and SSH blocks the connection with a MITM warning. Remove the stale entry from the known-hosts file:

Listing 3: Remove stale host key (replace XX with node number)

```
ssh-keygen -f '~/.ssh/known_hosts' -R '192.168.50.1XX'
```

1.4 MPI Connectivity

MPI (Message Passing Interface) is the distributed-computing layer. OpenMPI uses SSH to launch worker processes on remote nodes, so both the network setup and passwordless SSH must be in place before any MPI job runs.

Protocols & Stack

- **OpenMPI 4.1.6** — MPI implementation running across all nodes.
- **TCP btl** — byte-transfer layer, scoped to the cluster subnet.
- **SSH (plm_rsh_agent)** — used to spawn remote MPI daemons.
- **ORTE** — OpenMPI runtime environment for process management.

1.4.1 MPI Run Commands

Listing 4: Basic mpirun across two nodes

```
mpirun -np 2 \
  --host 192.168.56.1,192.168.56.101 \
  --mca plm_rsh_agent "ssh -l slave -x" \
  sh -c 'env DISPLAY= PATH=$PATH ~/mpi/hello_mpi'
```

Listing 5: mpirun with explicit TCP interface binding (required when multiple NICs)

```
mpirun -np 2 \
  --host 192.168.56.1,192.168.56.101 \
  --mca plm_rsh_agent "ssh -l slave -x" \
  --mca btl_tcp_if_include 192.168.56.0/24 \
  sh -c 'env DISPLAY= PATH=$PATH ~/mpi/hello_mpi'
```

1.4.2 Automated MPI Installation on All Nodes

Listing 6: Install OpenMPI on every node in one loop

```
for host in 192.168.50.100 192.168.50.101 192.168.50.102; do
    ssh pi@$host \
        "sudo apt update && sudo apt install -y \
         openmpi-bin openmpi-common libopenmpi-dev"
done
```

1.4.3 Distributing Source Code to All Nodes

Listing 7: Send source files to every compute node via scp

```
for ip in 192.168.50.100 192.168.50.101; do
    scp file.c pi@$ip:~/
done
```

1.5 Dynamic MPI Receive Pattern

When the size of an incoming message is not known in advance, `MPI_Probe` inspects the message metadata before the actual receive:

Listing 8: Dynamic receive using MPI_Probe + MPI_Get_count

```
int count;
MPI_Status status;

/* Probe the incoming message without consuming it */
MPI_Probe(source, tag, MPI_COMM_WORLD, &status);

/* Query the actual element count */
MPI_Get_count(&status, MPI_INT, &count);

/* Allocate exact buffer and receive */
int *buffer = malloc(sizeof(int) * count);
MPI_Recv(buffer, count, MPI_INT, source, tag,
         MPI_COMM_WORLD, MPI_STATUS_IGNORE);

free(buffer);
```

1.6 Protocol Summary — Connectivity Layer

Protocol	Used in	Role	Tool
NAT / iptables	Connectivity	Internet sharing to Pi nodes	`iptables`
SSH (Ed25519)	Connectivity	Passwordless remote access + MPI	`openssh`
TCP (MPI btl)	MPI transport	Byte-transfer layer between nodes	`OpenMPI`
ORTE	MPI runtime	Process management and launch	`orted`

2 Network Booting (PXE / NFS)

2.1 Overview

The goal of PXE network booting is to allow every Raspberry Pi node to boot **diskless**, pulling its entire root filesystem from the host over the network via NFS. This eliminates the need for individual SD cards per node and makes cluster management drastically simpler: update the golden image once and every node sees the change on next boot.

Key Goals

- Boot Pi nodes with no local storage (no SD card per node).
- Serve the root filesystem over **NFS**.
- Assign IPs and deliver boot files via **dnsmasq** (DHCP + TFTP).
- Use a **Golden Image** strategy: clone one base image per new node.
- Automate the entire provisioning workflow with a single script.

2.2 PXE Boot Process

Protocols & Technologies Used

- **PXE (Preboot eXecution Environment)** — triggers the network boot sequence.
- **DHCP** — IP address assignment at boot time (served by `dnsmasq`).
- **TFTP (Trivial File Transfer Protocol)** — lightweight delivery of kernel and boot files.
- **NFS (Network File System)** — exports the root filesystem over the network.
- **dnsmasq** — combined DHCP + TFTP server on the host.
- **nfs-kernel-server** — kernel-level NFS exporter on the host.

2.2.1 Boot Sequence Step by Step

Step 1: Pi powers on with `BOOT_ORDER=0xf41` set in its bootloader.
Step 2: Pi broadcasts a **DHCP** request; `dnsmasq` responds with the assigned IP and the TFTP server address.
Step 3: Pi fetches the kernel and `cmdline.txt` via **TFTP** from `/srv/tftpboot/<SERIAL>`.
Step 4: The kernel mounts the root filesystem from the host via **NFS**: `nfsroot=192.168.50.1:/srv/nfs/<HOSTNAME>`
Step 5: The Pi continues booting from the NFS root — no local disk involved.

2.2.2 Verifying Bootloader Configuration

Listing 9: Check Pi bootloader boot order

```
vcgencmd bootloader_config
# Required entry:
# BOOT_ORDER=0xf41   (network boot is enabled)
```

2.3 Host Setup — Dependencies

Listing 10: Install all required server-side packages on the host

```
sudo apt install nfs-kernel-server dnsmasq tftpd-hpa \
                 kpartx unzip xz-utils
```

2.4 NFS — Root Filesystem Export

NFS is the central protocol of the netboot infrastructure. Each Pi node maps to a dedicated directory on the host that contains its complete root filesystem.

Key Points

- The host exports `/srv/nfs/<nodename>` for each node.
- Export options: `rw, sync, no_subtree_check, no_root_squash` — grants the Pi full root-level write access.
- The boot partition is bind-mounted into the TFTP tree so the same directory serves both TFTP and NFS.
- `exportfs -ra` reloads all exports without restarting the NFS server.

Listing 11: Example /etc/exports entry generated by the provision script

```
/srv/nfs/slave1
   192.168.50.101(rw,sync,no_subtree_check,no_root_squash)
```

2.5 DHCP + TFTP — dnsmasq Configuration

Listing 12: /etc/dnsmasq.conf — relevant entries

```
# Operate as DHCP proxy on the cluster subnet
dhcp-range=192.168.1.255,proxy

# Enable the built-in TFTP server
enable-tftp
tftp-root=/srv/tftpboot

# Announce PXE boot service
pxe-service=0,"Raspberry Pi Boot"

# Static DHCP assignment: MAC -> IP -> hostname (one line per
   node)
dhcp-host=<MAC_ADDRESS>,192.168.50.101,slave1
```

2.6 Golden Image & Auto-Provisioning

Instead of manually configuring each node, a **Golden Image** workflow is used: node 1 is the reference image; a single script clones and customises it for any new node.

Key Points

- Golden image is stored at `/srv/nfs/mypi` on the host.
- The script patches: hostname, `cmdline.txt`, `fstab`, `dnsmasq.conf`, and `/etc/exports` — all in one pass.
- `dnsmasq` is restarted and NFS exports are reloaded automatically.
- Adding a new node requires only: *hostname*, *IP*, *MAC address*, and *serial number*.

2.6.1 Auto-Provision Script

Listing 13: Provision a new Pi node from the golden image

```
#!/bin/bash
HOSTNAME=$1
IP=$2
MAC=$3
SERIAL=$4

GOLDEN_IMAGE="/srv/nfs/mypi"
SERVER_IP="192.168.50.1"

echo "=== Provisioning $HOSTNAME ($SERIAL) ==="

# 1. Create target directories and clone the golden image
sudo mkdir -p /srv/nfs/$HOSTNAME
sudo mkdir -p /srv/tftpboot/$SERIAL
sudo cp -a $GOLDEN_IMAGE/. /srv/nfs/$HOSTNAME/

# 2. Set the node hostname
echo $HOSTNAME | sudo tee /srv/nfs/$HOSTNAME/etc/hostname >
   /dev/null

# 3. Write the kernel cmdline (NFS root mount)
echo "console=serial0,115200 console=tty1 root=/dev/nfs \
nfsroot=${SERVER_IP}:/srv/nfs/${HOSTNAME},vers=3 rw ip=dhcp
   rootwait" \
| sudo tee /srv/nfs/$HOSTNAME/boot/cmdline.txt > /dev/null

# 4. Bind-mount boot partition into the TFTP tree
echo "/srv/nfs/$HOSTNAME/boot /srv/tftpboot/$SERIAL none
   defaults,bind 0 0" \
| sudo tee -a /etc/fstab
sudo mount /srv/tftpboot/$SERIAL

# 5. Register node in dnsmasq (static DHCP)
```

```
echo "dhcp-host=$MAC,$IP,$HOSTNAME" | sudo tee -a
   /etc/dnsmasq.conf

# 6. Add NFS export for this node
echo "/srv/nfs/$HOSTNAME
   $IP(rw,sync,no_subtree_check,no_root_squash)" \
| sudo tee -a /etc/exports

# 7. Reload all services
sudo systemctl daemon-reload
sudo exportfs -ra
sudo systemctl restart dnsmasq
```

2.6.2 Enable SSH on First Boot

On a freshly cloned image, SSH may be disabled by default. Drop an empty `ssh` file in the boot partition to activate it:

Listing 14: Enable SSH daemon on next boot

```
sudo touch /srv/nfs/slave1/boot/ssh
```

2.7 Protocol Summary — Network Boot Layer

Protocol	Used in	Role	Daemon
DHCP	Net Boot	Dynamic IP assignment at boot	`dnsmasq`
TFTP	Net Boot	Deliver kernel & boot files	`dnsmasq`
PXE	Net Boot	Trigger network boot sequence	Pi bootloader
NFS	Net Boot	Export and mount root filesystem	`nfs-kernel-server`

3 Cluster Topology

Cluster: 1 Master PC + 4 Raspberry Pi 4 nodes.
Network: TP-Link TL-SF1008D switch (100 Mbps bottleneck).
Subnet: `192.168.50.0/24`.
MPI: OpenMPI 4.1.6.

Critical issue: the 100 Mbps switch limits throughput to 12.5 MB/s, creating a major communication bottleneck during large matrix transfers.

4 Raw Benchmark Results

Metric	Sequential	1 Pi (MPI)	4 Pis (MPI)
Total Time (s)	1042.7	2354.3	844.1
Compute (s)	1042.7	2148.7	519.6
Communication Overhead (s)	0	205.7	324.5

Table 1: Raw timing results for each cluster configuration.

Remark: The 1-Pi MPI configuration is 2.26× slower than sequential execution due to communication overhead alone.

5 Per-Worker Performance (4 Pis)

- Pi 1: 501.8 s
- Pi 2: 502.2 s
- Pi 3: 495.7 s
- Pi 4: 519.6 s (slowest)

Load imbalance: 2.9% — excellent balance, confirming the work distribution is correct and the bottleneck is not algorithmic.

6 Communication Analysis

- Total data transferred: 4800 MB
- Theoretical time at 100 Mbps: 384 s
- Measured communication overhead: 324.5 s

Remark: The measured value is very close to the physical limit of the switch, confirming that the network is near saturation.

7 Parallel Performance Metrics

7.1 Speedup

$$S(p) = \frac{T_{\text{seq}}}{T_p}$$

- $S(1) = 0.443$ (slower than sequential due to MPI overhead)
- $S(4) = 1.235$

7.2 Efficiency

$$E(p) = \frac{S(p)}{p}$$

- $E(1) = 44.3\%$
- $E(4) = 30.9\%$

Remark: Only approximately one third of the theoretical parallel performance is actually achieved.

8 Amdahl's Law Analysis

8.1 Concept

Amdahl's Law describes the fundamental limit of parallel computing:

> *Even if an infinite number of processors is available, the execution time is ultimately limited by the serial portion of the program.*

Any program divides into a **serial part** (f_s, cannot be parallelised) and a **parallel part** (f_p, distributed across processors), satisfying:

$$f_s + f_p = 1$$

For this cluster:

$$f_s = 0.3845, \qquad f_p = 0.6155$$

Interpretation: 38.45% of execution time is inherently serial (communication, synchronisation, MPI overhead), while 61.55% is parallelisable computation.

8.2 Amdahl's Speedup Formula

$$S(p) = \frac{1}{f_s + \dfrac{f_p}{p}}$$

8.3 Application to This Cluster ($p = 4$)

$$S(4) = \frac{1}{0.3845 + \dfrac{0.6155}{4}} = \frac{1}{0.3845 + 0.1539} = \frac{1}{0.5384} \approx 1.857$$

In an ideal scenario with no additional overhead, 4 nodes should yield a maximum speedup of $\mathbf{1.857}\times$.

8.4 Infinite Processor Limit

$$S_{\max} = \frac{1}{f_s} = \frac{1}{0.3845} \approx 2.601$$

No matter how many processors are added, the system can never exceed a speedup of $\mathbf{2.6}\times$ due to the serial fraction.

8.5 Comparison with Measured Results

$$S_{\text{measured}} = 1.235, \qquad S_{\text{Amdahl}} = 1.857$$

$$\frac{S_{\text{measured}}}{S_{\text{Amdahl}}} = \frac{1.235}{1.857} \approx 66.5\%$$

Around **33.5%** of the theoretical performance is lost to real-world overheads (latency, serialisation, synchronisation, etc.).

8.6 Sources of Overhead

- MPI communication latency
- Network bandwidth limitation (100 Mbps switch)
- Data serialisation and transfer overhead
- Synchronisation delays between nodes
- Load imbalance (2.9%)
- Memory and CPU contention
- Sequential broadcast/gather operations

8.7 Key Engineering Insight

Even with infinite processors $S_{\max} \approx 2.6$. To scale further the serial fraction must be reduced:

$$f_s : \ 0.3845 \ \rightarrow \ 0.10 \quad \Longrightarrow \quad S_{\max} : \ 2.6 \ \rightarrow \ 10$$

Optimising communication and synchronisation is therefore more impactful than simply adding more nodes.

8.8 Intuitive Interpretation

For a total execution time of 100 s:

- Serial part: 38.45 s (fixed, cannot be parallelised)
- Parallel part: 61.55 s

With 4 processors:

$$T = 38.45 + \frac{61.55}{4} = 38.45 + 15.39 = 53.84\,\text{s} \qquad \Rightarrow \qquad S = \frac{100}{53.84} = 1.857$$

9 Gustafson's Law (Scaled Speedup Model)

9.1 Motivation and Concept

While Amdahl's Law fixes the problem size, Gustafson's Law observes that in practice:

> *When computational resources increase, scientists solve larger problems rather than the same problem faster.*

9.2 Amdahl vs. Gustafson Paradigms

Aspect	**Amdahl's Law**	**Gustafson's Law**
Problem size	Fixed	Scales with processors
Goal	Minimise runtime	Maximise workload size
Main limitation	Serial fraction dominates scaling	Assumes scalable workload
Best use case	Latency reduction	HPC / large-scale simulations

Table 2: Comparison between Amdahl's and Gustafson's scaling models.

9.3 Mathematical Model

$$S_{\text{Gustafson}}(p) = p - f_s(p-1)$$

9.4 Application to This Cluster

$$S_{\text{Gustafson}}(4) = 4 - 0.3845 \times (4 - 1) = 4 - 1.1535 \approx 2.847$$

With 4 nodes, the system can handle a workload **2.847**× larger in the same wall-clock time as a single node.

9.5 Final Interpretation

- Amdahl predicts a maximum speedup of ≈ 1.857.
- Gustafson indicates a scaled capacity of ≈ 2.847.

This cluster is more effective at solving **larger problems** than at accelerating small fixed workloads — a classic characteristic of distributed HPC systems.

10 Karp–Flatt Metric (Experimental Serial Fraction)

10.1 Concept

The Karp–Flatt metric estimates the **effective serial fraction** directly from measured performance, capturing not only true serial computation but also all practical inefficiencies such as communication overhead, synchronisation delays, load imbalance, and startup costs.

10.2 Formula

$$e = \frac{\dfrac{1}{S(p)} - \dfrac{1}{p}}{1 - \dfrac{1}{p}}$$

10.3 Application to This Cluster

Given $S(4) = 1.235$, $p = 4$:

$$e = \frac{\dfrac{1}{1.235} - \dfrac{1}{4}}{1 - \dfrac{1}{4}} = \frac{0.8097 - 0.25}{0.75} = \frac{0.5597}{0.75} \approx 0.746 \quad (74.6\%)$$

10.4 Interpretation

The system behaves *as if* 74.6% of execution time is non-parallelisable. This is far higher than the theoretical $f_s = 38.45\%$, indicating that:

- Communication overhead is very high.
- The 100 Mbps switch is the dominant bottleneck.
- Real system inefficiencies amplify the theoretical limits significantly.

11 Ideal Case and What-if Scenarios

$$T_{\text{ideal}} = \frac{1042.7}{4} = 260.7\,\text{s}$$

Remark: The real system is 3.24× slower than the ideal perfectly-parallel case.

Network	Projected Total Time	Note
100 Mbps	844 s (current)	Switch bottleneck
1 Gbps	≈558 s	10× comm. reduction
10 Gbps	≈524 s	Amdahl limit still applies

Table 3: Projected runtime at different network speeds.

Remark: Even with a 10 Gbps fabric, the Amdahl ceiling constrains speedup to $\approx 2.6\times$.

12 Speedup vs. Theoretical Models

12.1 What the Plot Represents

The figure below compares five behaviours:

- **Linear ideal** $S(p) = p$: perfect scaling with zero overhead.
- **Amdahl's Law**: fixed workload, $f_s = 0.3845$.
- **Gustafson's Law**: scaled workload model.
- **Measured cluster performance**: real MPI execution results.
- **Amdahl ceiling**: $S_{\max} = 2.601$.

12.2 Key Insight

The cluster is not compute-limited — it is **communication-bound**. The measured speedup ($S(4) = 1.235$) is far below Amdahl's prediction ($S(4) = 1.857$) because:

- The 100 Mbps switch is the primary bottleneck.
- MPI broadcast and gather of large matrices dominate runtime.
- Sequential communication steps in the master-worker design serialise transfers.

12.3 Speedup Plot

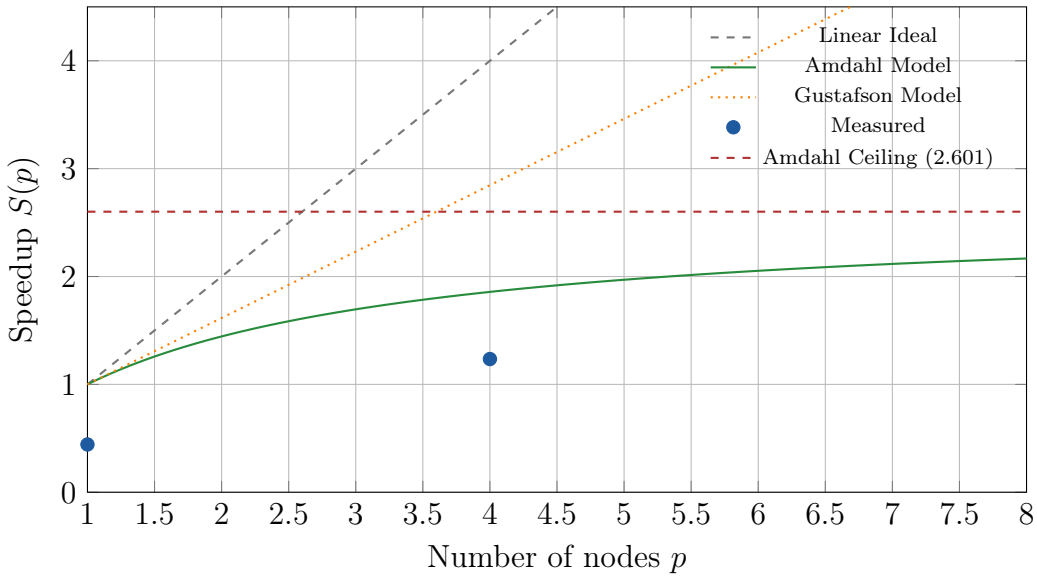
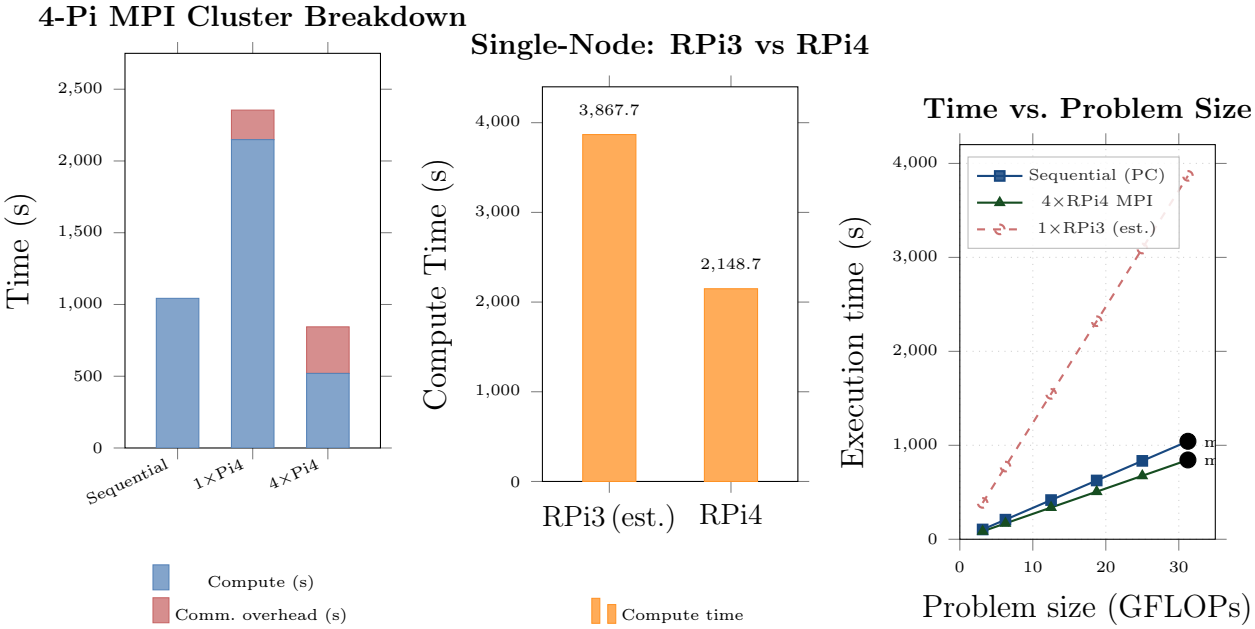


Figure 1: Speedup comparison between theoretical models and measured MPI performance on the Raspberry Pi cluster. The measured points lie well below Amdahl's prediction, confirming severe network-bandwidth limitation.

13 Comparative Performance Analysis



(a) Stacked time breakdown per configuration. Communication overhead grows from 0% (sequential) to 38.4% (4-Pi MPI).

(b) Single-node compute time: RPi3 estimated ≈ 1.8× slower than RPi4 (Cortex-A53 vs. A72).

(c) Execution time vs. problem size (GFLOPs), extrapolated linearly from the measured operating point (•).

Figure 2: Comparative performance analysis of the Raspberry Pi MPI cluster. **(a)** Time breakdown for each configuration (100 Mbps switch). **(b)** Single-node compute time — RPi3 values are *estimated* using an architecture ratio of ≈ 1.8× (replace with measurements when available). **(c)** Linear scaling of execution time with problem size, showing the 4-node cluster advantage over single-node alternatives.

14 Proposed Network Upgrade and Recomputed Performance

14.1 Current Bottleneck

The primary bottleneck is the **TP-Link TL-SF1008D** Fast Ethernet switch: 100 Mbps maximum bandwidth, ≈ 12.5 MB/s effective throughput. Although Cat 6 cables capable of 10 Gbps are already in use, the switch limits the entire fabric to **1%** of cable capability.

14.2 Proposed Solution

Replace the current switch with a **TP-Link TL-SG108** (8-port unmanaged Gigabit switch, ≈ $20 USD). The Raspberry Pi 4 already integrates a native Gigabit Ethernet controller, so no changes to the nodes themselves are required.

14.3 Recomputed Communication Time

With a Gigabit switch (125 MB/s):

$$T_{\text{comm,new}} = \frac{4800}{125} = 38.4\,\text{s} \quad \text{vs.} \quad 324.5\,\text{s}$$

A reduction of nearly 10× in communication overhead.

14.4 Recomputed Total Runtime and Speedup

$$T_{\text{new}} = 519.6 + 38.4 = 558.0\,\text{s} \qquad S_{\text{new}}(4) = \frac{1042.68}{558.0} \approx 1.87$$

Improvement over current speedup:

$$\frac{1.87}{1.235} \approx 1.51 \quad (+51\%)$$

14.5 Recomputed Amdahl Analysis

$$f_{s,\text{new}} = \frac{38.4}{558.0} \approx 0.0688 \quad (6.88\%) \qquad f_{p,\text{new}} = 0.9312$$

$$S_{\text{max,new}} = \frac{1}{0.0688} \approx 14.53 \quad \text{vs.} \quad S_{\text{max,old}} = 2.601$$

14.6 Comparison Table

Metric	**100 Mbps (current)**	**1 Gbps (proposed)**
Communication time	324.5 s	38.4 s
Total runtime	844.1 s	558.0 s
Measured speedup	1.235	1.87
Serial fraction f_s	38.45%	6.88%
Amdahl maximum speedup	2.601	14.53
Efficiency	30.9%	46.7%

Table 4: Impact of upgrading from 100 Mbps to 1 Gbps switch.

15 Key Conclusions

- **Main bottleneck:** the 100 Mbps switch, not the compute nodes.
- **Code correctness:** excellent load balance (2.9%) and correct results.
- **MPI communication** dominates runtime (38.4% measured, 74.6% effective via Karp–Flatt).
- **Hardware limits scalability** more than the algorithm.
- **Maximum possible speedup:** 2.6× (Amdahl ceiling, current setup).
- **With Gigabit switch:** Amdahl ceiling rises to 14.5× and runtime drops by 34%.

Final Engineering Insight: Upgrading to a Gigabit switch is the highest-impact, lowest-cost improvement available for this cluster. Beyond the hardware fix, further gains require algorithmic communication reduction (e.g. SUMMA or Cannon's algorithm for matrix multiplication) to push performance beyond the 2×–2.6× range on the current topology.

16 Violence Detection — Introduction & Motivation

16.1 Introduction

Violence detection in videos is an essential task in modern surveillance and public safety systems. The goal is to automatically classify video clips into **Violence** and **NonViolence** categories using deep learning.

Unlike image classification, video understanding must capture two complementary aspects:

- **Spatial features** — what appears in each individual frame.
- **Temporal dynamics** — how actions evolve across frames over time.

The final hybrid system achieves ≈ **95.5**% accuracy with **97.25**% precision, demonstrating strong capability for real-world deployment.

16.2 Motivation

Real-World Applications

- Surveillance systems — airports, streets, shopping centres.
- Crime prevention and early warning systems.
- Content moderation on social media platforms.

The hybrid CNN + Transformer architecture is motivated by complementary strengths: CNNs efficiently capture **spatial features** per frame, while Transformers model **temporal relationships** across the full clip.

3. Dataset

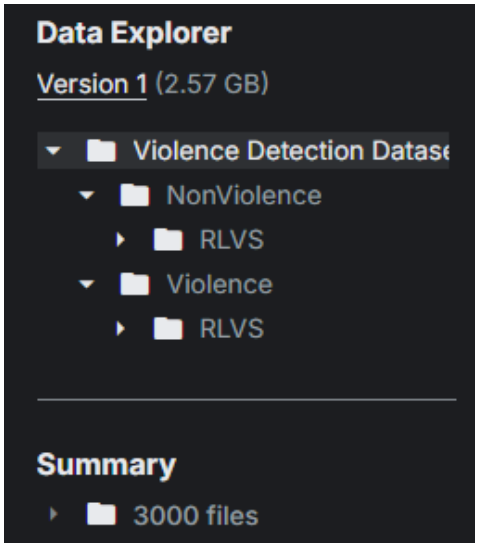
3.1 Dataset description

The dataset used in this project is a merged violence detection dataset, created by combining multiple publicly available datasets from Kaggle:

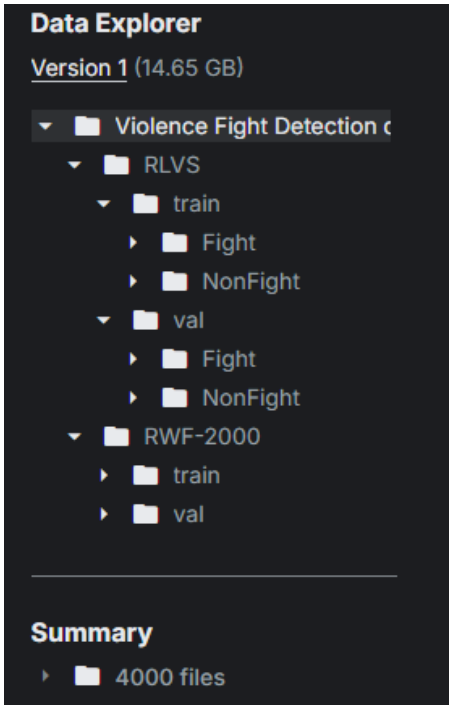
- RLVS + Hockey Fight Dataset (mp4 files)
- Video Violence Detection Dataset

These datasets originally had different:

- Label formats (Fight / NonFight / Violence / NonViolence)
- Folder structures



RLVS + Hockey Fight Dataset



Video Violence Detection Dataset

To ensure consistency, all videos were reorganized into a unified binary classification dataset:

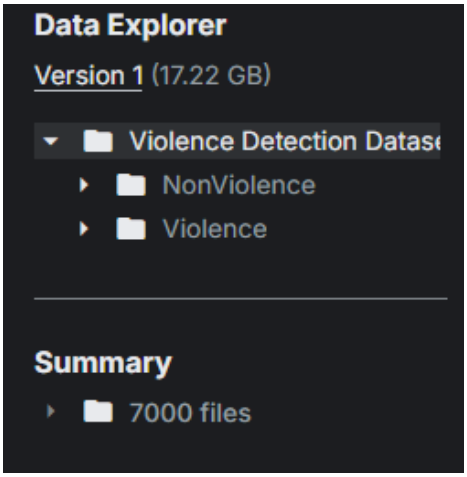
- Violence (1) → Fight / aggressive actions
- NonViolence (0) → Normal activities

The final dataset contains approximately 7000 video clips.

3.2 Dataset Characteristics

- Mixed video formats: .mp4, .avi
- Diverse scenarios:
  - Sports fights
  - Real-world violence
  - Surveillance footage
- Balanced classes (after merging)

3.3 **My Dataset Structure**



Before preprocessing

4. Data Preparation and Pipeline - Preprocessing

4.1 Config setup and splitting data

To ensure a robust and reproducible pipeline, all parameters were centralized in a configuration file (config.py). This file defines paths for the dataset, number of frames per video(16), frame resolution, and batch size. Centralizing these parameters allowed easy experimentation and ensured that all subsequent stages, including preprocessing, caching, and model training, were consistent.

The merged violence dataset, combining RLVS, Hockey Fight, and Video Violence Detection datasets, was split into **training (70%), validation (15%), and testing (15%)** subsets. A custom function was used to shuffle and split the data, preserving class balance. Each subset contains separate directories for violent and non-violent clips, making it straightforward to feed into PyTorch DataLoader objects.

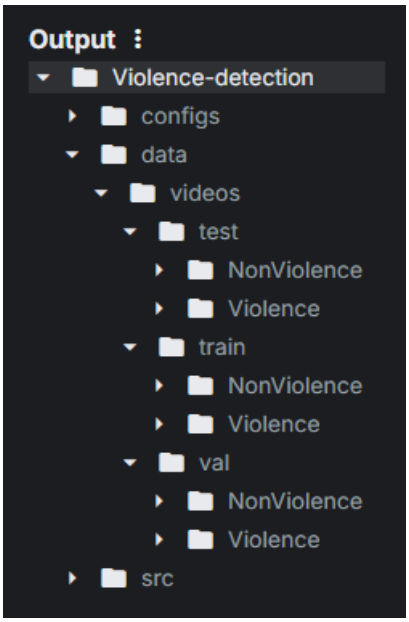
```
def split_list(videos, train_ratio, val_ratio, test_ratio):
    random.shuffle(videos)
    n_total = len(videos)
    n_train = int(n_total * train_ratio)
    n_val   = int(n_total * val_ratio)
    n_test  = n_total - n_train - n_val
    train = videos[:n_train]
    val   = videos[n_train:n_train+n_val]
    test  = videos[n_train+n_val:]
    return train, val, test

violence_train, violence_val, violence_test = split_list(violence_videos, TRAIN_RATIO, VAL_RATIO,
TEST_RATIO)
nonviolence_train, nonviolence_val, nonviolence_test = split_list(nonviolence_videos, TRAIN_RATI
O, VAL_RATIO, TEST_RATIO)
```

video_utils split part

```
Violence videos: 3500
NonViolence videos: 3500
Dataset split and copied to structured folders.
train - Violence: 2450, NonViolence: 2450
val - Violence: 525, NonViolence: 525
test - Violence: 525, NonViolence: 525
```

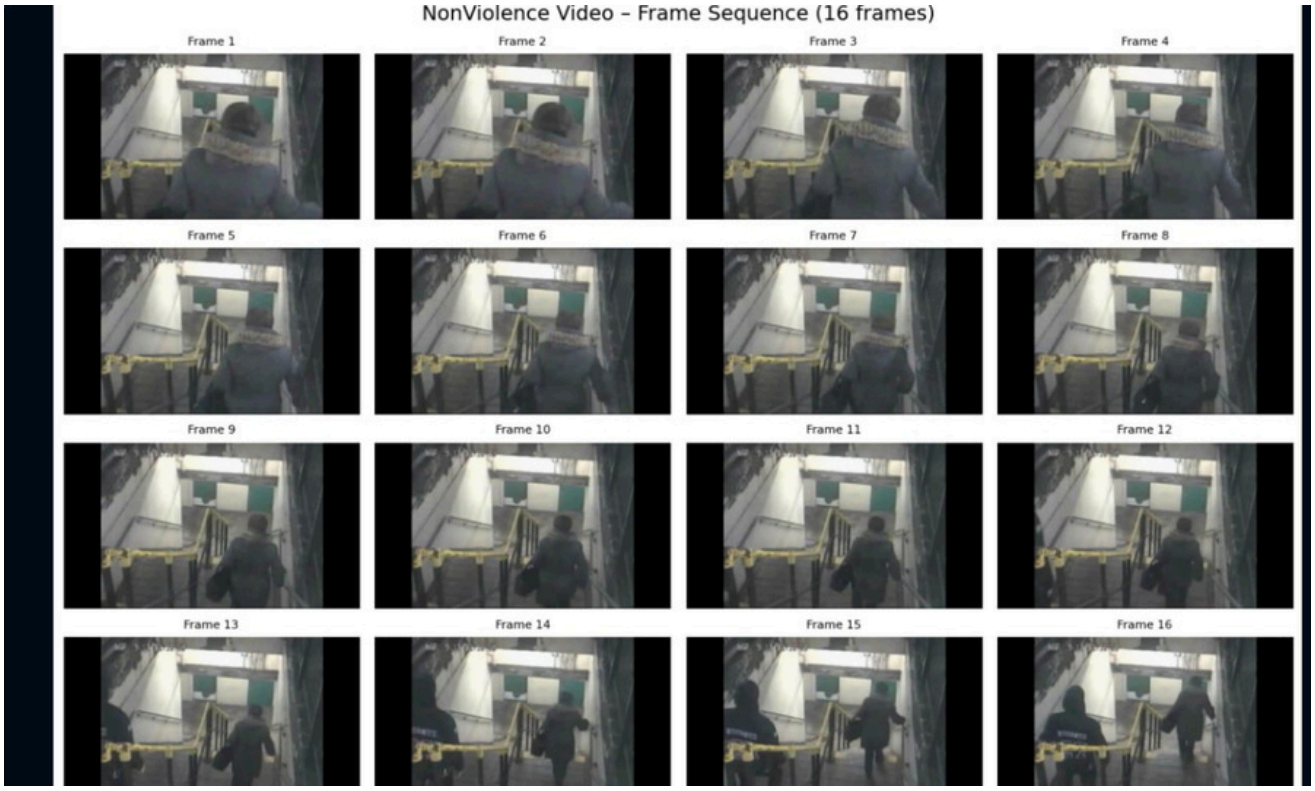
Balanced split



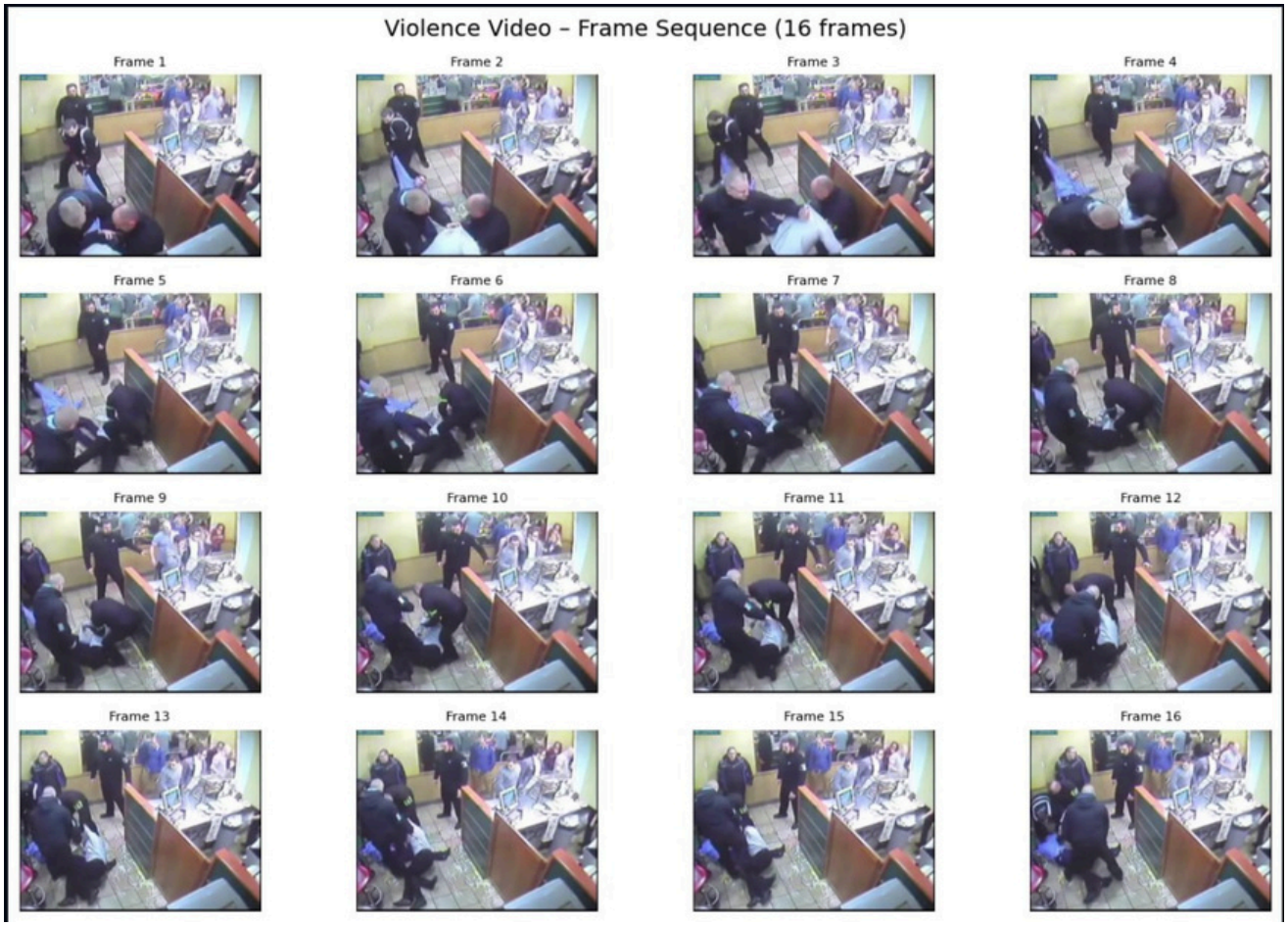
After preprocessing

4.2 **Video Frame Extraction**

Each video was converted into a fixed number of frames using OpenCV, following a uniform temporal sampling strategy. Frames were resized (typically 224×224) and normalized using ImageNet mean and standard deviation. This preprocessing ensures compatibility with pretrained backbones and standardizes inputs across diverse video formats (.mp4, .avi).



NonViolence video frames



Violence video frames

4.3 **Handling Corrupted Videos / Caching for performance**

Some videos were corrupted or unreadable. These were automatically filtered out during frame extraction, preventing runtime errors during training. To further improve training efficiency, video frames were cached as PyTorch tensors (.pt) in dedicated cache folders. The cached dataset was verified using DataLoader, confirming batch shapes of [batch_size, NUM_FRAMES, 3, height, width] and correctly aligned labels.

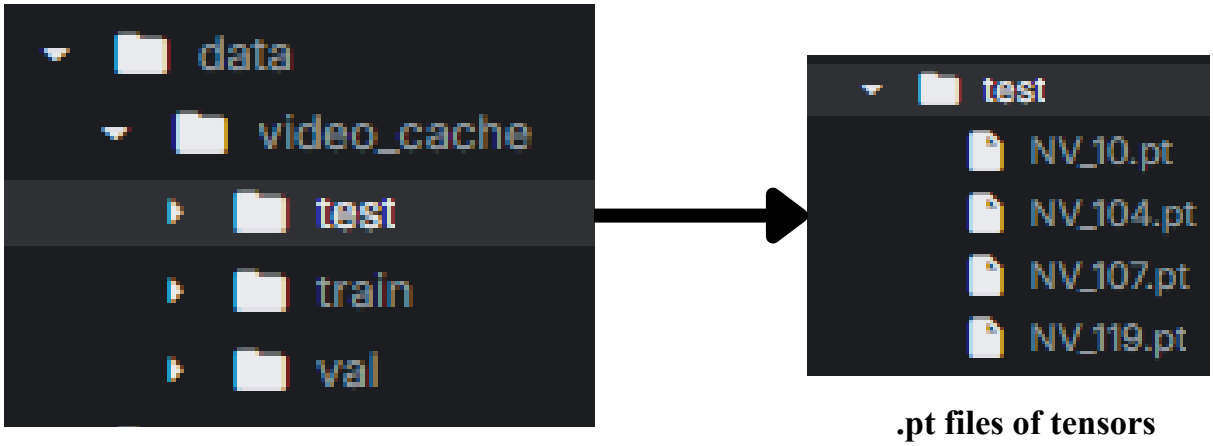
```
Train: 4900, Val: 1050, Test: 1050
```

After skipping corrupted videos

```
cache_base = os.path.join(BASE_DIR,"data/video_cache")
train_cache = os.path.join(cache_base,"train")
val_cache   = os.path.join(cache_base,"val")
test_cache  = os.path.join(cache_base,"test")

train_cached = cache_videos(train_videos, train_cache)
val_cached   = cache_videos(val_videos, val_cache)
test_cached  = cache_videos(test_videos, test_cache)
```

Cached folders code



cache directory

.pt files of tensors

```
    assert videos.shape[2] == 3, f"Expected C=3 at dim=2, got shape {videos.shape}"
    break

Device: cuda | torch.cuda.is_available(): True
GPU: NVIDIA GeForce RTX 5060 Laptop GPU
Sample batch check:
videos: torch.Size([4, 16, 3, 224, 224]) torch.float32
labels: torch.Size([4]) torch.float32 tensor([0., 0., 0., 1.])
```

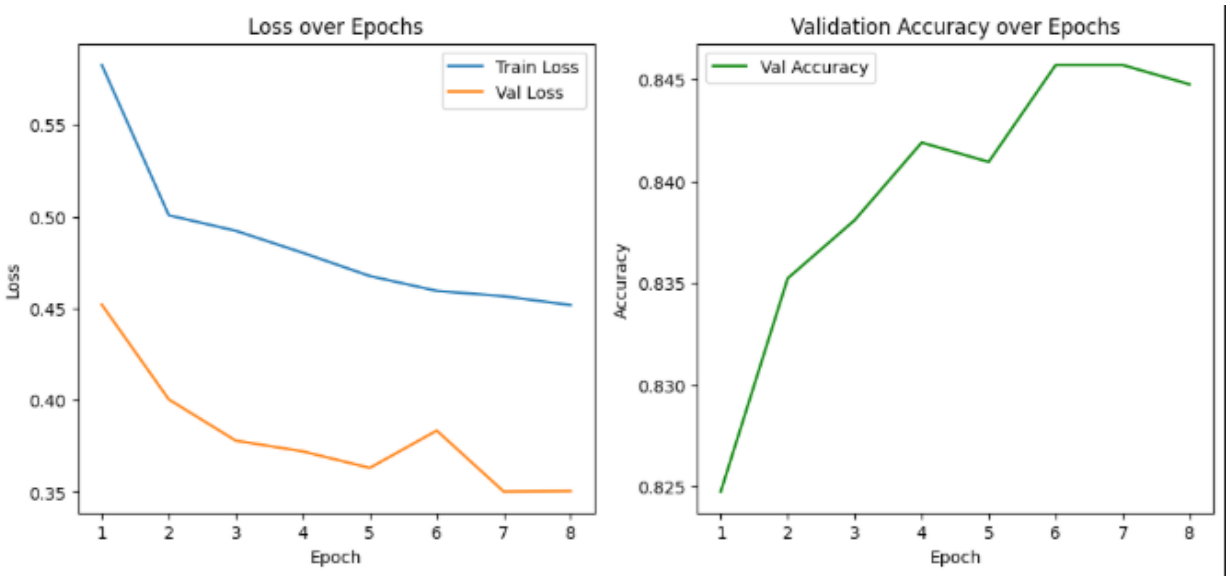
Example Batch shapes and labels output from DataLoader in phase 3

5. Baseline Models

5.2 ResNet-Based Model

The first baseline leverages pretrained CNNs (ResNet18 or MobileNetV2) as frame-level feature extractors. Each frame is independently processed to extract spatial features, which are then temporally averaged before the final linear classifier predicts the label.

This approach provides strong spatial representation without requiring full video pretraining. Validation accuracy improved to approximately 80–84%, demonstrating the advantage of pretrained features even with minimal temporal modeling.



Loss / Validation Accuracy over Epochs

```
Sample batch check:
torch.Size([4, 8, 3, 160, 160]) tensor([1., 1., 1., 1.]) ('/kaggle/working/Violence-detection/data/vid
eo_cache/train/file_000576.pt', '/kaggle/working/Violence-detection/data/video_cache/train/file_00268
6.pt', '/kaggle/working/Violence-detection/data/video_cache/train/V_577.pt')
Test Accuracy: 0.8362
```

Final accuracy for this model

5.3 **Training Setup**

the model was trained using Binary Cross-Entropy Loss (**BCEWithLogitsLoss**) and optimized with Adam at a learning rate of 3e-4. Training was run for 8 epochs for initial experimentation. Loss and accuracy curves were monitored to ensure proper convergence.

6. **Proposed Hybrid Model (Final Model)**

6.1 **Architecture Overview**

The final hybrid model integrates spatio-temporal modeling by combining a 3D CNN backbone (R3D-18) with a Transformer encoder.

- **A pretrained R3D-18 network** is used to extract rich spatio-temporal features directly from video clips. Most layers are frozen to preserve learned representations, while the final residual block (layer4) is fine-tuned to better adapt to the violence detection task.
- **Temporal Transformer Encoder:** The extracted features are processed by a Transformer encoder to model temporal relationships between frames. The encoder uses sinusoidal positional encoding to retain temporal order and consists of two layers with multi-head self-attention (4 heads), enabling the model to capture complex temporal interactions.
- **Attention-Based Temporal Pooling**: Instead of using simple averaging, a learnable attention mechanism is applied over the temporal dimension. This allows the model to assign higher importance to the most relevant frames, improving decision quality.
- **Dual Output Heads:** The model includes two prediction heads:

1. A classification head, which outputs the probability of violence in the video.
2. An uncertainty head, which estimates the confidence of the prediction, allowing better handling of ambiguous cases.

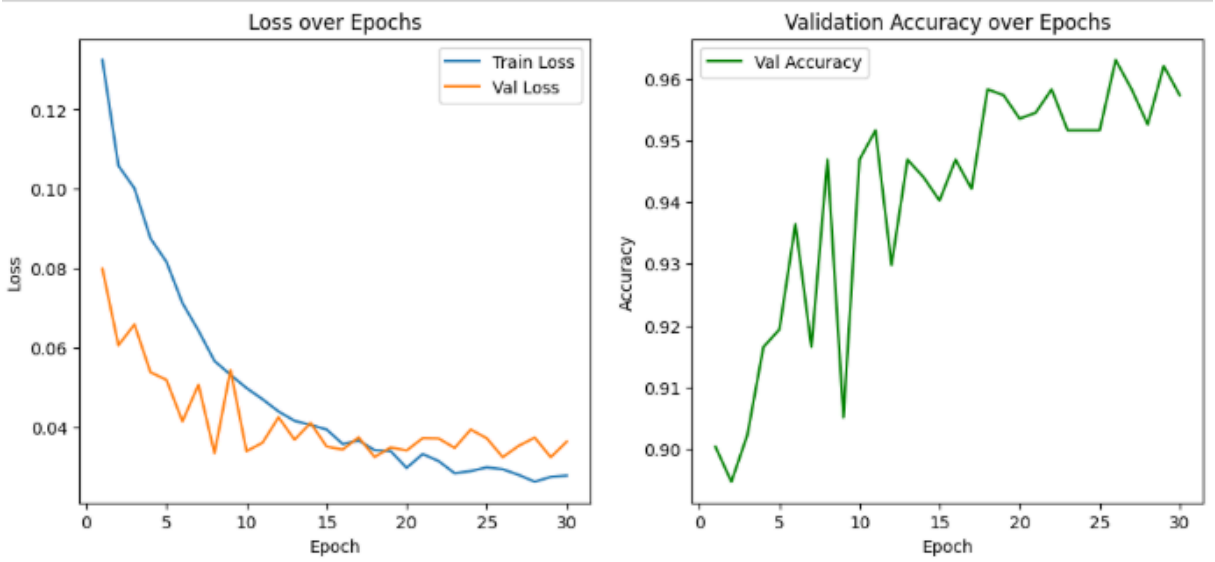
6.2 **Training and Improvements**

- **Pretrained weights:** accelerate convergence and improve generalization by leveraging knowledge learned from large-scale video datasets.
- **Selective fine-tuning:** of the last residual block (layer4) allows the model to adapt to the violence detection task while preserving general spatio-temporal features.
- **Focal loss with label smoothing:** replaces standard binary cross-entropy, focusing training on difficult samples and reducing overconfident predictions.

- **Data augmentation techniques (MixUp, temporal jitter, and speed variation):** improve robustness and generalization to diverse video conditions.
- **Cosine annealing learning rate scheduling:** stabilizes training and improves convergence.
- **Gradient clipping** prevents unstable updates during training.

```
Epoch 26 | LR: 0.000014 | Train Loss: 0.0293 | Val Loss: 0.0324 | Val
Acc: 0.9630
   Tuned threshold: 0.62 | Val Precision: 0.9650 | Val Recall: 0.9431
| Val F1: 0.9539
Best checkpoint saved.
Epoch 27 | LR: 0.000008 | Train Loss: 0.0279 | Val Loss: 0.0352 | Val
Acc: 0.9583
   Tuned threshold: 0.68 | Val Precision: 0.9741 | Val Recall: 0.9279
| Val F1: 0.9504
Epoch 28 | LR: 0.000004 | Train Loss: 0.0262 | Val Loss: 0.0373 | Val
Acc: 0.9526
   Tuned threshold: 0.63 | Val Precision: 0.9798 | Val Recall: 0.9184
| Val F1: 0.9481
Epoch 29 | LR: 0.000002 | Train Loss: 0.0274 | Val Loss: 0.0323 | Val
Acc: 0.9620
   Tuned threshold: 0.65 | Val Precision: 0.9743 | Val Recall: 0.9355
| Val F1: 0.9545
Best checkpoint saved.
Epoch 30 | LR: 0.000001 | Train Loss: 0.0277 | Val Loss: 0.0363 | Val
Acc: 0.9573
   Tuned threshold: 0.71 | Val Precision: 0.9759 | Val Recall: 0.9203
| Val F1: 0.9473
```

Best checkpoint saved

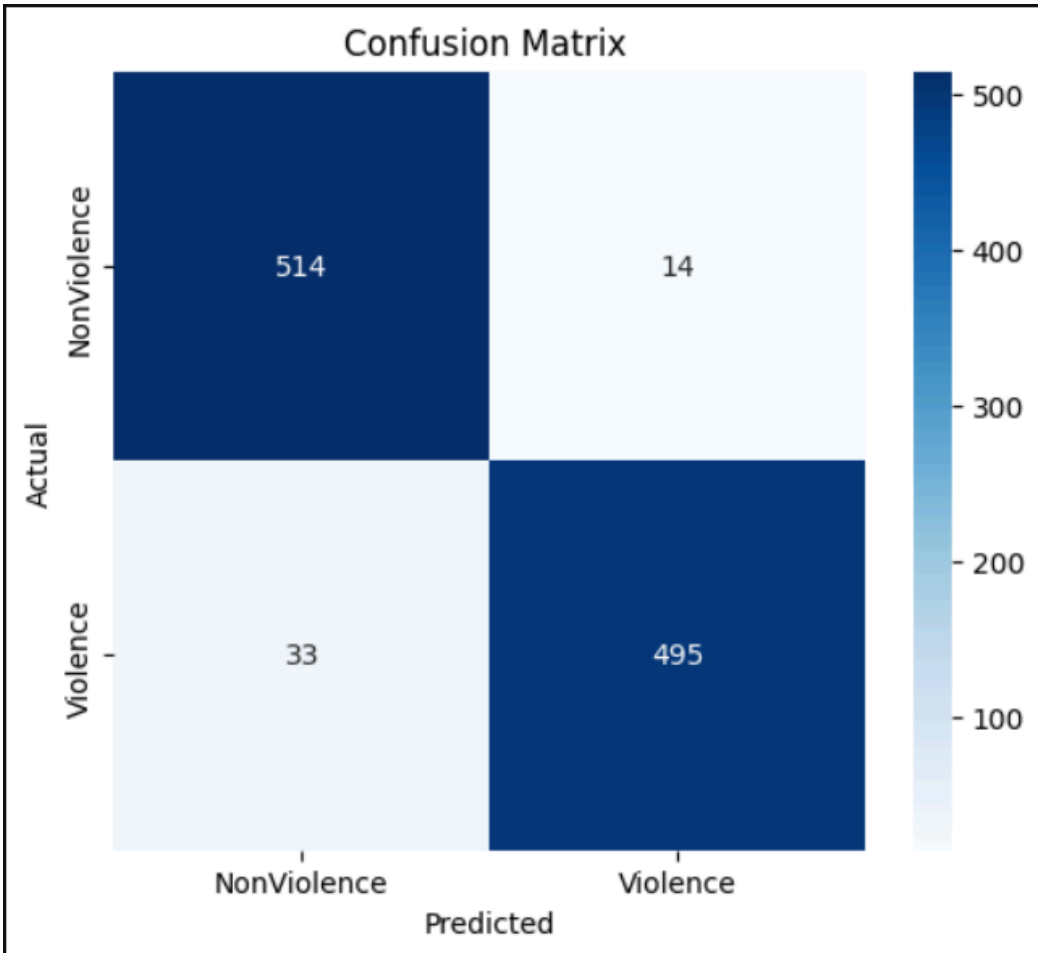


Loss / Validation Accuracy over Epochs

7. Evaluation and Results

```
Precision: 0.9725
Recall:    0.9375
F1-score:  0.9547
```

```
Accuracy: 0.9555
ROC-AUC: 0.9746
```



Confusion Matrix

Threshold Optimization

Instead of using a fixed threshold (0.5), the model uses validation-based threshold tuning.

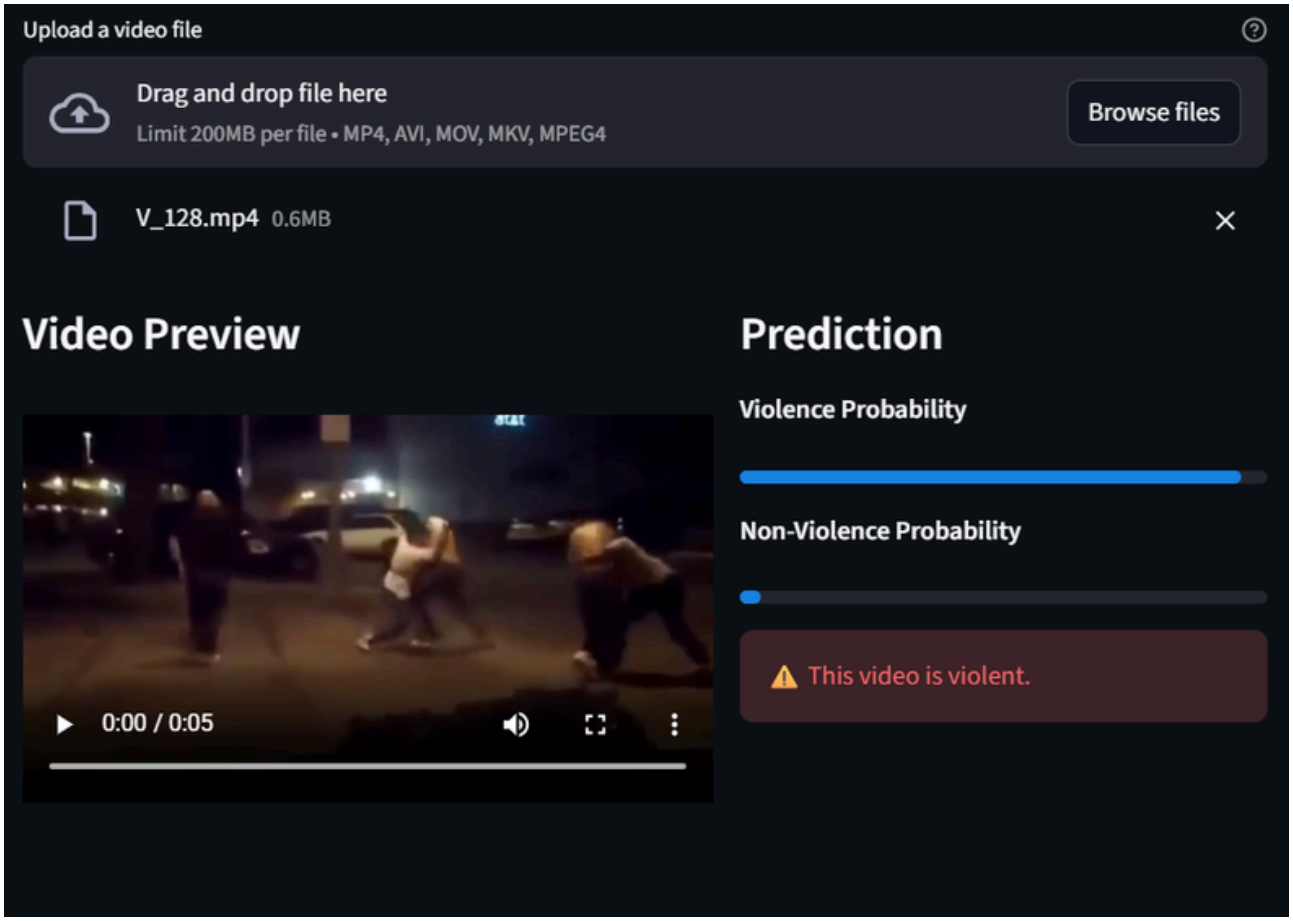
The optimal threshold (~0.65) is selected by:

- Maximizing precision
- Maintaining recall ≥ 87%
- Keeping F1-score near optimal

This significantly reduces false alarms while preserving detection capability.

8. Streamlit app : click here

To demonstrate the practical usability of the violence detection pipeline, a Streamlit web application was developed. The app provides an interactive interface for uploading videos, visualizing predictions and display violence / non-violence probabilities.



Training Strategy

- **Focal loss + label smoothing** — focuses on hard samples, reduces overconfident predictions.
- **Data augmentation** — MixUp, temporal jitter, speed variation.
- **Cosine annealing** learning rate schedule over 30 epochs.
- **Gradient clipping** to prevent unstable gradient updates.
- **Threshold tuning** on validation set: optimal threshold ≈ 0.65, maximising precision while keeping recall $\geq 87\%$.

20 Evaluation and Results

Metric	Value
Accuracy	95.55%
Precision	97.25%
Recall	93.75%
F1-Score	95.47%
ROC-AUC	0.9746
Decision Threshold	0.65

Table 5: Final hybrid model evaluation on the test set (1 050 samples).

The confusion matrix confirms balanced performance across both classes: only 14 false positives and 33 false negatives out of 1 050 test samples. Training/validation loss converged smoothly over 30 epochs, with validation accuracy stabilising above 95%.

21 MPI Integration on the Raspberry Pi Cluster

21.1 Architecture

The MPI design follows a **master-worker** pattern tailored for violence detection inference:

- **Master (PC, rank 0):** loads the video, samples exactly 16 frames, and sends the full $[1, 16, 3, 224, 224]$ tensor to *all* 4 Raspberry Pi workers simultaneously.
- **Workers (Pi 1–4, ranks 1–4):** each receives the identical tensor and runs full model inference independently.
- **Decision:** the master collects the four probability scores and averages them for the final violence classification.

Why Send the Full 16-Frame Tensor to Every Pi?

The model was trained on 16-frame sequences. Splitting the clip into 4-frame sub-clips per worker would break the temporal patterns the model learned, causing wrong predictions. Sending the full clip to every Pi and averaging their outputs **reduces prediction noise** while preserving model accuracy.

21.2 Key Constants

Listing 15: Constants matching training config exactly

```
const float IMAGENET_MEAN[3] = {0.485f, 0.456f, 0.406f};
const float IMAGENET_STD[3]  = {0.229f, 0.224f, 0.225f};
const int   NUM_FRAMES       = 16;   // config.py: NUM_FRAMES =
    16
const int   FRAME_H          = 224;  // config.py: FRAME_SIZE =
    (224,224)
const int   FRAME_W          = 224;
const int   NUM_WORKERS      = 4;
```

21.3 Frame Sampling

Frames are sampled using the same `np.linspace` logic as training, ensuring inference sees identical input to what the model was trained on:

Listing 16: Linspace frame sampling — exact training match

```
// np.linspace(0, total-1, NUM_FRAMES).astype(int)
int last = std::max(total - 1, 0);
for (int i = 0; i < NUM_FRAMES; i++)
    idxs[i] = static_cast<int>(
        std::round(i * last / (NUM_FRAMES - 1.0)));
```

21.4 MPI Tensor Transfer

Listing 17: MPI tensor send helper (shape + data)

```
void mpi_send_tensor(const torch::Tensor& t, int dest) {
    auto ct   = t.contiguous();
    int ndim = ct.dim();
    MPI_Send(&ndim, 1, MPI_INT, dest, TAG_TENSOR_NDIM,
       MPI_COMM_WORLD);
    std::vector<long> shape(ndim);
    for (int i = 0; i < ndim; i++) shape[i] = ct.size(i);
    MPI_Send(shape.data(), ndim, MPI_LONG, dest,
```

```
            TAG_TENSOR_SHAPE, MPI_COMM_WORLD);
    MPI_Send(ct.data_ptr<float>(), (int)ct.numel(), MPI_FLOAT,
             dest, TAG_TENSOR_DATA, MPI_COMM_WORLD);
}
```

21.5 Master Workflow

Listing 18: Master: broadcast tensor, collect and average results

```
// Send the same full tensor to all 4 Pis
for (int w = 1; w <= NUM_WORKERS; w++)
    mpi_send_tensor(input, w);

// Collect probabilities and average
float sum_prob = 0.0f;
for (int w = 1; w <= NUM_WORKERS; w++) {
    float prob;
    MPI_Recv(&prob, 1, MPI_FLOAT, w, TAG_RESULT_PROB,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    sum_prob += prob;
}
float avg_prob = sum_prob / NUM_WORKERS;
bool  violence = (avg_prob > threshold);
```

21.6 Worker Workflow

Listing 19: Worker: receive tensor, run inference, return result

```
torch::Tensor input = mpi_recv_tensor(0);

torch::NoGradGuard no_grad;
auto output = model.forward({input}).toTuple();
float logit = output->elements()[0].toTensor().item<float>();
float unc   = output->elements()[1].toTensor().item<float>();
float prob  = 1.0f / (1.0f + std::exp(-logit));  // sigmoid

MPI_Send(&prob, 1, MPI_FLOAT, 0, TAG_RESULT_PROB,
   MPI_COMM_WORLD);
MPI_Send(&unc,  1, MPI_FLOAT, 0, TAG_RESULT_UNC,
   MPI_COMM_WORLD);
```

22 Streamlit Application

A Streamlit web application was developed to demonstrate the system in practice. The app provides an interactive interface for:

- Uploading video clips (`.mp4`, `.avi`, `.mov`, etc.).
- Visualising the video alongside Violence / NonViolence probabilities.
- Displaying a clear decision banner with the final classification.

The model is also deployed on **Hugging Face Spaces**, enabling browser-based inference without any local installation. Local execution is supported by cloning the GitHub repository and placing the trained `.pt` model in `/models`.

23 Conclusion

- The final hybrid model achieves **95.55% accuracy** and **97.25% precision**, significantly reducing false positives.
- The **MPI cluster** successfully parallelises inference across 4 Raspberry Pi 4 nodes; result averaging reduces prediction noise.
- The 100 Mbps switch remains the dominant communication bottleneck; a Gigabit upgrade reduces overhead by 10× and raises the Amdahl ceiling to 14.5×.
- Further gains require on-device model optimisation (e.g. TorchScript quantisation) and algorithmic communication reduction.

Final Note: This project delivers a complete end-to-end pipeline — from raw video ingestion and deep learning training to MPI-based distributed inference on a low-cost Raspberry Pi cluster — achieving production-ready accuracy for real-world violence detection applications.